Revisão Técnica do Port do Sensor de *Aging* para Intel MAX 10

Estado atual, correções, melhorias, plano de trabalho e critérios de aferição

Objeto da revisão. Os três commits da branch `max10-de10lite-port` posteriores ao *hand-off* do repositório: `a09d497` (07/10/2026), `92c306b` (07/10/2026) e `b1f7635` (14/10/2026), todos restritos a `max10_port/`.

Método. Leitura integral do RTL, das restrições, da IP do PLL e dos relatórios do Quartus; confronto com o plano (`docs/plano-port-max10.tex`) e com o RTL de referência em Artix-7. Todas as afirmações quantitativas deste documento foram *medidas* — por compilação no Quartus Prime Lite 25.1std (mesma versão usada na implementação) e por simulação no Questa Starter 2025.2 com os modelos oficiais da Intel (ALTPLL e átomos `fiftyfivenm`) — e não estimadas.

Público. Equipe do port MAX 10 e coordenação. O texto pressupõe familiaridade com a implementação Xilinx/Vivado e explica os equivalentes no Quartus à medida que aparecem (resumo no Apêndice A).

Laboratório	LESC — Laboratório de Engenharia de Sistemas de Computação, UFC
Data	29 de outubro de 2026
Base	`max10-de10lite-port` @ `b1f7635`
Ferramentas	Quartus Prime Lite 25.1std.0 b1129; Questa Altera Starter 2025.2; Icarus Verilog
Elaboração	Revisão técnica conduzida com apoio de Claude Code (Anthropic)

Sumário

Sumário executivo **1**

1 Contexto e método **1**
1.1 O mecanismo do sensor em uma página 1
1.2 Testes realizados nesta revisão 2

2 O que foi implementado **3**
2.1 As duas correções posteriores estão certas 4

3 Como isso contribui para o objetivo do projeto **5**

4 O que precisa ser corrigido **5**

5 O que pode ser melhorado **11**
5.1 M1. Voltar ao *dual-adder* da referência, na cadeia de carry 11
5.2 M2. Estimar a borda por curva de probabilidade 11
5.3 M3. Dimensionar período e caminho juntos 12
5.4 M4. Confirmar cada passo de fase 12
5.5 M5. Mais sensores e mais régua 12
5.6 M6. Operação e instrumentação 12

6 Próximas tarefas e distribuição na equipe **12**
6.1 Princípio da divisão 12
6.2 Papéis 13
6.3 Sequência 13

7 Como aferir que as implementações estão corretas **14**
7.1 Referência já obtida para o nível L2 14
7.2 Ensaios em bancada (L4) 14

8 Limitações desta revisão **15**

A Glossário Vivado ↔ Quartus **16**

B Ambiente e comandos **16**

C Arquivos dos experimentos **16**

Sumário executivo

Veredito

A arquitetura do sensor de referência é implementável no MAX 10. Os dois pilares que motivavam dúvida estão demonstrados: (i) controle da lógica no nível da LUT, com preservação contra otimização e fixação de posicionamento; (ii) três clocks alinhados a partir de um mesmo VCO, com deslocamento dinâmico de fase de um deles — o mecanismo do auto-*tuning*. Uma simulação de laço fechado do topo, com o modelo oficial do PLL, calibra exatamente na contagem prevista pela análise de timing.

Mas o trabalho ainda está antes do Gate A

O que existe é uma *prova de conceito de bancada*. Em relação ao objetivo da branch, faltam ou estão errados: o laço não é contínuo (uma única medida por *reset*); a régua está em 178 ps quando o piso possível é 96 ps; a geometria caminho/*catcher* produz leitura *silenciosamente* errada quando o caminho fica rápido (defeito demonstrado em simulação); os testbenches não refletem o RTL; o posicionamento do sensor não é reprodutível; e os binários versionados vêm de um build quebrado. Nenhum desses pontos é um impedimento: todos têm correção conhecida e verificável, descrita na Seção 4.

O que permanece genuinamente incerto

O risco científico do projeto é quantitativo, não de viabilidade: o passo de fase do MAX 10 é $\approx$4,9$\times$ mais grosso que o do Artix-7 e o nó de 55 nm envelhece mais devagar. Se o degrau de envelhecimento alcançável supera o ruído de medida só se descobre em placa. A Seção 6 organiza o trabalho para chegar a esse número o quanto antes.

Como ler este documento. A Seção 1 resume o mecanismo do sensor e os testes realizados. As Seções 2 e 3 descrevem o que foi feito e o que isso já prova. A Seção 4 é o núcleo prático: cada problema com evidência, causa, correção passo a passo e forma de verificação. A Seção 5 trata de melhorias arquiteturais; a 6, da divisão do trabalho em paralelo; a 7, de como aferir que cada entrega está correta.

1 Contexto e método

1.1 O mecanismo do sensor em uma página

O sensor mede quanto *tempo sobra* (o *slack*) entre o instante em que um sinal termina de atravessar um caminho lógico e o próximo flanco de clock. A Figura 1 mostra a estrutura implementada no MAX 10, que é a mesma da referência em Artix-7.

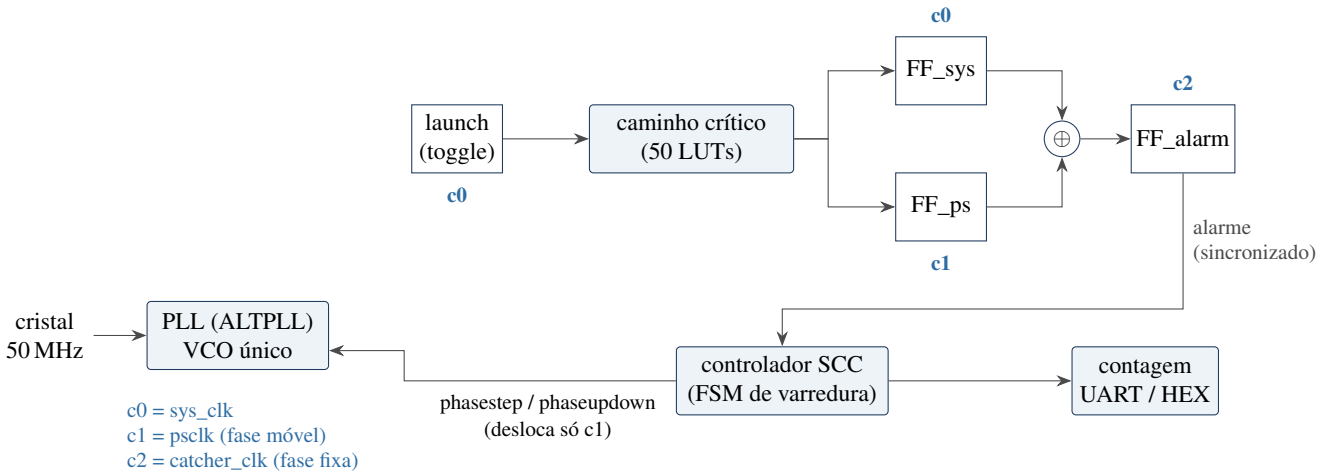


Figura 1: Estrutura do sensor. O PLL gera três clocks a partir do mesmo VCO; o controlador move a fase do psclk até que as duas capturas do caminho divirjam.

- O registrador `launch` inverte a cada ciclo de `sys_clk`, injetando uma transição no caminho a cada período T. A transição leva um tempo d para atravessar o caminho.
- `FF_sys` captura o fim do caminho no flanco de `sys_clk`; `FF_ps` captura o mesmo sinal no flanco de `psclk`, que o controlador *adianta* em passos de tamanho Δ (o "passo da régua").
- Enquanto o flanco do psclk ocorre depois de o sinal estabilizar, as duas capturas coincidem e o XOR é 0. Quando o adiantamento $\delta = k\Delta$ ultrapassa o *slack* $T - d$, `FF_ps` passa a capturar o valor antigo, o XOR vai a 1 e o `FF_alarm` (no `catcher_clk`) registra o alarme.
- O número de passos k dado até o alarme é a medida: $k\Delta \approx T - d$. Envelhecimento aumenta d, logo k diminui ao longo do tempo.

Por que a régua é confiável

O passo Δ é uma fração fixa do período do VCO, que é travado no cristal da placa. O caminho medido envelhece; a régua não. No Artix-7, $\Delta = T_{\mathrm{VCO}}/56 \approx 19{,}8\,\mathrm{ps}$; no MAX 10, $\Delta = T_{\mathrm{VCO}}/8$, com piso de $\approx 96\,\mathrm{ps}$ (VCO a 1300 MHz).

1.2 Testes realizados nesta revisão

A Tabela 1 lista os experimentos que fundamentam as conclusões. Os arquivos de cada um estão catalogados no Apêndice C e servem de ponto de partida para a suíte de testes do repositório.

Tabela 1: Experimentos executados (Quartus 25.1std Lite e Questa Starter 2025.2).

#	Experimento	Resultado
T1	Compilação do projeto como está	Sucesso; 308 LEs; VCO 700 MHz; passo 178 ps; slack de c0 = 170,6 ns. Controlador presente (no build versionado ele havia sido otimizado para constante).
T2	Simulação do ALTPLL (modelo Intel) com pulsos de `phasestep`	$-178{,}6$ ps por passo em C1 com `phaseupdown=0`; seleção `011`=C1 e `100`=C2 confirmadas. `phasedone` fica baixo por *meio* ciclo do scanclk.
T3	PLL pedindo passos de 96, 100 e 120 ps	VCO 1300 MHz/96 ps (M=26); 1250 MHz/100 ps; 1050 MHz/119 ps. O piso é atingível.
T4	*Back-annotation* de posicionamento e dois builds limpos	570 atribuições em nível de LC/FF; posicionamento idêntico entre builds.
T5	Sensor travado $\times$ livre, alterando o RTL vizinho	Livre: atraso do caminho mudou 6,8 ns ($\approx$70 passos de 96 ps). Travado: posições idênticas; atraso de setup variou 21 ps, de hold 208 ps.
T6	Fixar roteamento na edição Lite	Impossível: *back-annotation* de roteamento não existe para MAX 10; partições incrementais são removidas pela licença Lite.
T7	Somador 16 bits em cadeia de carry, sem trava, 3 variantes vizinhas	Atraso crítico 6,414 / 6,396 / 6,346 ns (dispersão de 68 ps).
T8	Netlist para simulação com timing (SDF)	Não disponível para MAX 10 (“functional netlist is the only supported type”).
T9	Laço fechado: topo + PLL oficial + cadeia como atraso do STA	Calibra em $\lfloor (T-d)/\Delta \rfloor + 1$ (primeiro passo além da borda) exatamente para d = 29,2; 20; 12; 10,5 ns.
T10	Idem, com caminho abaixo de 10 ns	Satura em 1065 passos independentemente de d (defeito do *catcher*, Seção 4).
T11	Testbenches versionados	`max10_aging_top_tb.sv` não compila; `scc_controller_tb.sv` falha em 3 testes e acusa 4 “violações”.

2 O que foi implementado

Os três commits criaram, praticamente do zero, o projeto Quartus em `max10_port/hdl/`. A Tabela 2 descreve cada componente, sua correspondência com a referência Vivado e sua situação.

Tabela 2: Componentes implementados.

Componente	O que faz (e equivalente Vivado)	Situação
`aging_pll.v` (IP ALTPLL)	50 MHz → c0/c1/c2 a 5 MHz; c2 com +10 ns fixos; c1 com deslocamento dinâmico. Equivale ao `clk_wiz_0` (MMCM com PS dinâmico).	Funciona; VCO não está no máximo.
`critical_path_chain.sv`	50 inversores instanciados como átomo `fiftyfivenm_lcell_comb` (`lut_mask=16'h5555`, `dont_touch`). Equivale a instanciar `LUT1`/`LUT3_L` com `DONT_TOUCH`.	Funciona: 50 LCs preservados.
`aging_sensor_core.sv`	FF_sys, FF_ps, XOR, FF_alarm. Equivale ao `modern_sensible`.	Correto; sem atributos de preservação.
`scc_controller.sv`	FSM: espera *lock*, sincroniza alarme, filtra, dá passos de fase por tempo, para no alarme. Equivale ao `controller_controller`.	Funciona, mas é *one-shot*.
`uart_tx.sv`, `uart_packet_sender.sv`	Pacote de 5 bytes a cada 100 ms, 115200 bps: `0xAA`, contagem, status, XOR. Equivale ao `sensor_stream`.	Funciona; protocolo frágil.
`hex7seg.sv` + topo	Mostra a contagem em 4 displays; 7 LEDs de depuração.	Funciona.
`max10_aging.sdc`	Clock de 50 MHz, `derive_pll_clocks`, *false paths* do sensor.	Funciona; comentários desatualizados.
`max10_aging_top.qsf`	Pinos da DE10-Lite, 3 FFs fixados em LAB, SignalTap ligado.	Gerado pela GUI.
Testbenches (`src/sim/`)	Teste do controlador e de integração.	Desatualizados (T11).
Saídas de build	`db/`, `incremental_db/`, `output_files/` (`.sof`, `.pof`), bibliotecas de simulação.	Versionadas por engano; de um build quebrado.

2.1 As duas correções posteriores estão certas

`92c306b` — seleção do contador e *lock*. No MAX 10, `phasecounterselect` usa 000 = todos, 001 = M, 010 = C0, 011 = C1, 100 = C2. O valor anterior (`100`) deslocava o *catcher*, não o psclk. O mesmo commit ligou `pll_locked` ao controlador; sem isso a porta fica em GND, o controlador permanece em *reset* e o Quartus o remove inteiro ("*stuck at GND*" em `map.rpt`). Ambas confirmadas em T1 e T2.

`b1f7635` — direção do deslocamento. Com `phaseupdown=1` o psclk *atrasa*: o alarme só existiria se $\delta > d$, mas o *catcher* amostra em +10 ns, antes de o FF_ps atualizar, e nunca o vê — a fase "passa direto". Com `phaseupdown=0` o psclk *adianta* (como `psincdec=0` na referência) e mede-se $T - d$. Confirmado em T2 e T9.

O diagnóstico do *handshake* também estava certo. O comentário [C] do controlador atribui o travamento ao pulso curto de `phasedone` e passa a sequenciar o *handshake* por tempo. T2 mostra que, no modelo da Intel, `phasedone` fica baixo apenas entre o flanco de subida e o de descida do scanclk: uma FSM que só amostra em subida nunca o observa. A solução por tempo é legítima; a melhoria possível (confirmar cada passo) está na Seção 5.

3 Como isso contribui para o objetivo do projeto

O plano define quatro requisitos para o port. A Tabela 3 mostra o quanto cada um avançou com a implementação atual somada aos testes desta revisão.

Tabela 3: Progresso em relação aos requisitos do plano (Seção 1.1 do plano).

Requisito	Estado	O que já está provado / o que falta
1. Medir o *slack* com régua rastreável e caracterizada	Parcial	Provado: régua travada no cristal, passo em simulação e no fit. Falta: piso de 96 ps, medição do passo em placa, DNL.
2. Laço de auto-*tuning* operacional	Parcial	Provado: varredura até o alarme, em laço fechado com o PLL oficial (T9). Falta: varredura contínua com retorno de fase, reaquisição, detecção de faixa.
3. Fidelidade em ps com incerteza declarada	Não iniciado	Depende de série temporal (req. 2) e de hardware.
4. Arquitetura independente de fabricante	Evidência favorável	Os mecanismos-chave portam (LUT, preservação, posicionamento, PLL dinâmico). A cadeia de inversores difere do *dual-adder* RCA de referência; precisa de decisão documentada.

O valor real do que foi feito

O maior risco técnico de qualquer port é descobrir tarde que a ferramenta não deixa controlar a estrutura do circuito. Esse risco foi eliminado: a implementação atual demonstra preservação de LUTs, PLL com deslocamento dinâmico e um laço que converge. O que resta são problemas de engenharia com solução conhecida e a pergunta científica de detectabilidade.

4 O que precisa ser corrigido

Cada item traz: o problema, a evidência, por que importa, como corrigir e como verificar a correção. A ordem é a de prioridade; os itens C1–C4 bloqueiam o Gate A.

C1. Repositório não reproduzível e binários de um build quebrado — Crítico

Um clone limpo não compila, e o `.sof` versionado programa um sensor inoperante.

Evidência

- `output_files/*.sof/.pof` e `db/` datam de 06/10, antes da correção do `pll_locked`; nesse build o controlador inteiro foi otimizado para constante.
- 194 arquivos e ≈15 MB de saídas de ferramenta, arquivos `.bak`, caminhos absolutos de Windows nos relatórios.
- O QSF ativa o SignalTap com `output_files/stp1.stp`, que não está versionado, e aponta um testbench em `../../../../../../../Downloads/`.
- `src/constraints/max10_aging.qsf` é um QSF órfão e divergente; `scripts/create_project.tcl` continua um *template*.

Por que importa

Sem um build reproduzível a partir de texto versionado, ninguém consegue verificar o que outra pessoa fez, e qualquer medida em placa fica sem rastreabilidade de qual circuito foi medido.

Como corrigir

1. Acrescentar ao `.gitignore` da raiz as regras do Quartus:

```
# Quartus Prime (max10_port)
max10_port/hdl/db/
max10_port/hdl/incremental_db/
max10_port/hdl/output_files/
max10_port/hdl/greybox_tmp/
max10_port/hdl/simulation/
max10_port/hdl/build/
max10_port/hdl/*.qws
max10_port/hdl/*.rpt
*.bak
*.bak[0-9]
```

2. Remover do índice (sem apagar a cópia local de quem estiver trabalhando):

```
git rm -r --cached max10_port/hdl/db max10_port/hdl/incremental_db \
  max10_port/hdl/output_files max10_port/hdl/simulation max10_port/hdl/
      greybox_tmp
git rm --cached max10_port/hdl/*.bak max10_port/hdl/src/**/*.bak max10_port/
    hdl/*.qws
```

3. Manter versionados apenas: RTL, SDC, a IP (`aging_pll.v`, `.qip`, `.ppf`) e *um* QSF. Recomenda-se que o QSF passe a ser gerado por `scripts/create_project.tcl` (preenchendo os TODOs com as listas de arquivos e atribuições), como nos projetos Vivado; o QSF da GUI vira saída.
4. Desligar o SignalTap no QSF de referência (`ENABLE_SIGNALTAP OFF`); quem precisar dele liga localmente.
5. Trazer o testbench de gate-level para `src/sim/` e corrigir o caminho, ou remover as linhas `EDA_TEST_BENCH_*`.
6. Para distribuir bitstreams, seguir a convenção do Nexys: uma pasta `hdl/bitstreams/` com o `.sof` *e* o hash do commit de origem no nome.
7. Fixar a versão: registrar em `hdl/README.md` "Quartus Prime Lite 25.1std.0 Build 1129" e exigir essa versão de toda a equipe.

Como verificar

Em um clone limpo: `scripts/check_layout.sh`, depois `scripts/create_project.sh` e `scripts/build_bitstream.sh` terminam sem erro; o `map.rpt` não contém "*stuck at GND*" para registradores do `scc_controller`; `git status` fica limpo após o build.

C2. O laço de auto-tuning não é contínuo — **Crítico**

Uma medida por *reset* manual: impossível obter série temporal, σ_{resid} ou rastrear deriva.

Evidência

No controlador atual, o estado `S_DONE` é terminal (linhas 249–251 de `scc_controller.sv`). A referência, `controller_controller.sv`, trava o valor, devolve a fase a zero e recomeça sozinha.

Por que importa

É o requisito 2 do plano e o critério do Gate A ("laço rastreando uma borda deslocada"). Toda a análise estatística depende de muitas medidas sucessivas.

Como corrigir

Acrescentar à FSM os estados de retorno de fase da referência, preservando o *handshake* temporizado que já funciona (Figura 2).

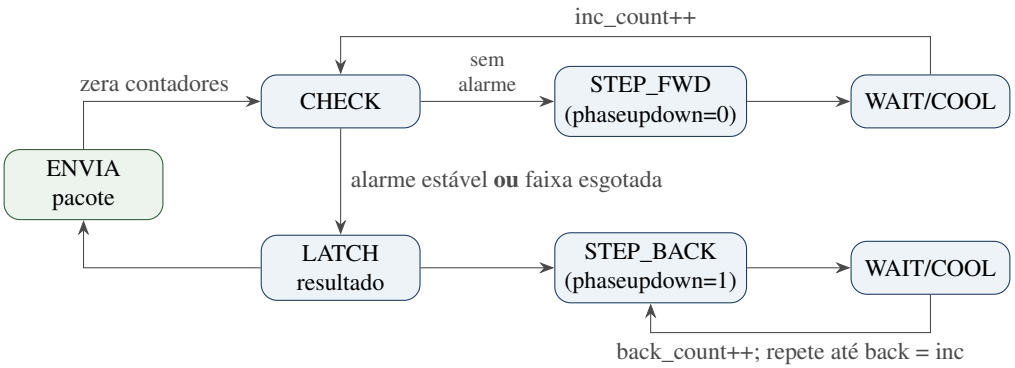


Figura 2: FSM contínua proposta. O retorno é feito pelo mesmo número de passos dados para frente, de modo que cada varredura começa sempre da fase zero.

Esboço dos pontos novos (a lógica de *timer* do *handshake* é a atual, reaproveitada nos dois sentidos):

```
// phaseupdown passa a ser controlado pela FSM: 0 = adianta (medicao), 1 = atrasa
     (retorno)
assign phaseupdown = (dir == DIR_BACK);

S_CHECK: if (alarm_stable || inc_count == MAX_STEPS) next_state = S_LATCH;
       else next_state = S_STEP_FWD;
S_LATCH: next_state = (inc_count == 0) ? S_RESTART : S_STEP_BACK;
// S_STEP_BACK/S_WAIT_BACK: mesmo handshake temporizado; back_count++ ao fim de
    cada passo
S_WAIT_BACK_DONE: next_state = (back_count == inc_count) ? S_RESTART :
    S_STEP_BACK;
S_RESTART: begin inc_count <= 0; back_count <= 0; seq <= seq + 1; next_state =
    S_CHECK; end

// no S_LATCH: result <= inc_count; result_valid <= 1;
// out_of_range <= (inc_count == MAX_STEPS); // borda nao encontrada
```

Pontos de atenção:

- `MAX_STEPS` deve ser um parâmetro de topo derivado de T e Δ (uma volta completa: T/Δ); se atingido, a medida é marcada inválida, nunca reportada como número.
- O resultado é *registrado* em `S_LATCH` e permanece estável durante o retorno; o pacote UART lê sempre o último resultado válido, com número de sequência.
- Após o retorno, aguardar alguns ciclos de `catcher_clk` antes do próximo `CHECK` para o alarme da varredura anterior se dissipar (zerar o filtro).

Como verificar

Testbench com o modelo oficial do PLL e cadeia comportamental (base pronta, T9): (a) em d fixo, 20 varreduras consecutivas devolvem a mesma contagem; (b) mudando d durante a simulação (degrau de $+500$ ps), a contagem seguinte cai de $\lceil 500/\Delta \rceil$ ou $\lfloor 500/\Delta \rfloor$ passos; (c) após cada retorno, a fase medida de c1 em relação a c0 volta a 0 ps; (d) com caminho impossível (sem alarme), a FSM marca `out_of_range` e recomeça, sem travar.

C3. Geometria caminho/catcher: leitura errada sem aviso — **Crítico**

Se o caminho ficar mais rápido que o atraso do *catcher*, o sensor passa a medir o *catcher*.

Evidência

T10: com a cadeia abaixo de $\approx$10 ns, a contagem satura em 1065 passos ($1065 \times 178{,}6\,\text{ps} \approx 190\,\text{ns} = T - 10\,\text{ns}$), qualquer que seja o atraso real. O corner rápido do STA dá $d \approx 10{,}5$ ns: margem de apenas

≈0,5 ns.

Por que acontece

A Figura 3 mostra a janela de tempo. O XOR só reflete a comparação correta entre o flanco de `sys_clk` (quando FF_sys atualiza) e o próximo flanco de psclk (quando FF_ps atualiza de novo). O *catcher*, em $+c$ após `sys_clk`, precisa cair dentro dessa janela:

$$t_{co} + t_{xor} \;<\; c \;<\; T - \delta.$$

No limiar de alarme, $\delta = T - d$, então a condição vira $c < d$. Se $c > d$, o psclk adiantado "passa" por cima do *catcher* antes de o caminho falhar, e o alarme dispara em $\delta = T - c$: a leitura vira $\min(T-d,\ T-c)$.

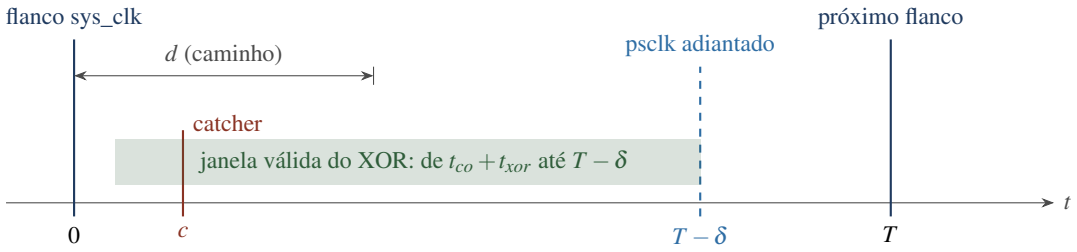


Figura 3: Janela de amostragem. A medida só é válida enquanto o *catcher* (c) estiver antes de $T-\delta$. Como o alarme legítimo ocorre em $\delta = T - d$, é preciso $c < d$ com folga em todos os corners.

Como corrigir

1. **Regra de projeto**, a ser cumprida em todos os corners de PVT e ao longo do envelhecimento:

$$t_{co} + t_{xor} + \text{margem} \;<\; c \;<\; d_{\min} - \text{margem}, \qquad d_{\max,\text{envelhecido}} < T - \text{margem}.$$

2. **Reduzir** c: hoje são 10 ns; bastam ≈2–3 ns (o suficiente para FF_sys atualizar e o XOR assentar). Alterar `clk2_phase_shift` na IP.
3. **Garantir por STA o lado inferior**: substituir os dois `set_false_path` de clk[0]/clk[1] para clk[2] por uma restrição que obrigue o XOR a assentar antes do *catcher*, por exemplo

```
# o XOR precisa estar estavel antes do catcher (c), com folga
set_max_delay -from [get_registers {*ff_sys_clk_q *ff_psclk_q}] \
              -to [get_registers {*ff_alarm_q}] 1.500
```

   e conferir no relatório que o caminho fecha.
4. **Dimensionar o caminho** (ver também Seção 5): com $T = 200$ ns e $d \approx 29$ ns, só 15% do período é caminho, o que aproxima $d_{\min}$ de c e desperdiça sinal de envelhecimento (que é proporcional a d). Escolher T e o comprimento do caminho juntos, pela regra acima.
5. **Detector de saturação** no RTL: se o resultado ficar a menos de alguns passos de $(T-c)/\Delta$, marcar a medida como inválida no pacote. Isso transforma um erro silencioso em um erro visível.
6. **Modo de diagnóstico em placa**: o `phasecounterselect` também alcança C2, então o *catcher* pode ser varrido dinamicamente para localizar, no silício, a fronteira de saturação.

Como verificar

Repetir T9/T10 com os novos parâmetros: para d variando do corner rápido ao lento (acrescido de, por exemplo, 10% de envelhecimento), a contagem deve seguir $\lfloor (T-d)/\Delta \rfloor + 1$ em todos os pontos e o detector de saturação deve disparar apenas quando $d < c$. No STA, o caminho FF_sys/FF_ps→FF_alarm deve fechar contra a restrição do item 3.

C4. A régua está em 178 ps, não no piso de 96 ps — **Alto**

Perda de 1,86× de resolução sem nenhum ganho, e uma constante errada no código.

Evidência

`fit.rpt`: VCO 700 MHz, M=14, passo 178 ps. O comentário do topo diz que 96 ps seriam "do artigo de referência (MMCM da Xilinx)" — é o contrário: 96 ps é o piso do MAX 10; o MMCM dá 19,8 ps. O código usa 179 ps; o nominal é 178,57 ps.

Como corrigir

1. Regenerar a IP ALTPLL pedindo passo de 96 ps (no assistente da IP, ou alterando `vco_phase_shift_step = 96` e regenerando). T3 mostra que o Quartus fecha com VCO 1300 MHz, M=26.
2. Conferir no `fit.rpt` (seção *PLL Summary*) o valor *efetivo*, e não o pedido.
3. Tornar o passo e a faixa parâmetros de topo (`STEP_PS`, `MAX_STEPS`) e enviar `STEP_PS` no pacote de telemetria, para o host nunca usar uma constante copiada à mão.
4. O número que vale é o *medido em placa* (Seção 7, ensaio H2).

Como verificar

`fit.rpt` mostrando 1300 MHz/96 ps; simulação de T2 refeita mostrando −96,15 ps por passo; ensaio H2 em placa dentro de ±2% do nominal.

C5. Testbenches desatualizados — **Alto**

Nenhum teste versionado valida o RTL atual; mudanças futuras não terão rede de segurança.

Evidência

T11: o teste de integração instancia portas que não existem mais; o do controlador não liga `pll_locked`, verifica o *handshake* antigo e espera `phaseupdown=1`.

Como corrigir

Reescrever a suíte em três níveis, todos executáveis por um único script (`scripts/run_sim.sh`) e com resultado **PASS/FAIL** automático (sem inspeção de forma de onda):

1. **Unitários (Icarus ou Questa)**: controlador com modelo de borda probabilística (base pronta em `edge_tb.sv`); UART com decodificador de pacotes e checagem de CRC.
2. **Integração (Questa)**: topo com o modelo oficial do ALTPLL (`-L altera_mf_ver`) e a cadeia substituída por atraso comportamental parametrizado com os valores do STA (base pronta em `sim/top/`).
3. **Átomos (Questa)**: a cadeia real com `-L fiftyfivenm_ver`, para verificar a lógica dos `lut_mask`.

Lembrete: o MAX 10 não tem simulação com timing (T8). O atraso usado na simulação de integração deve vir do STA, e o script deve extraí-lo automaticamente do relatório.

Como verificar

O script retorna código 0 só se todos os testes passam; cada correção C2, C3, C6 e C7 entra acompanhada do teste que falhava antes e passa depois.

C6. Filtro de alarme medido no clock errado — **Médio**

"16 amostras consecutivas" equivale a só 1–2 observações independentes.

Evidência

O filtro conta ciclos de 50 MHz (scanclk), mas o alarme só muda a cada ciclo do `catcher_clk` (5 MHz): cada valor é contado ≈10 vezes.

Como corrigir

Contar no domínio do *catcher* (amostras consecutivas de `ff_alarm_q`) e sincronizar só o resultado; ou, no domínio do controlador, contar apenas quando um *strobe* sincronizado indicar uma nova amostra. Expressar o parâmetro em amostras do *catcher*: `ALARM_FILTER_SAMPLES`.

Como verificar

Teste unitário com alarme intermitente de probabilidade conhecida no último passo antes da borda: a taxa de parada prematura deve bater com p^N para N amostras exigidas.

C7. Protocolo de telemetria frágil — Médio

Sem número de sequência, sem enquadramento robusto, alarme não sincronizado.

Evidência

O byte de sincronismo `0xAA` pode aparecer na carga útil; o checksum é XOR de 3 bytes; `alarm` entra no domínio da UART sem sincronizador (o `byte3` e o checksum podem divergir); não há sequência, versão nem campo de validade.

Como corrigir

Definir um *pacote v2* documentado (proposta):

Bytes	Campo	Observação
0–1	`0xA5 0x5A`	cabeçalho de 2 bytes
2	versão do protocolo	começa em 2
3–4	sequência (u16, LE)	detecta perda de pacotes
5–6	contagem (u16, LE)	último resultado válido
7	flags	válido, saturado, fora de faixa, alarme, *lock*
8	erros de passo (u8)	passos sem `phasedone` confirmado
9–10	`STEP_PS` (u16)	passo nominal configurado
11–12	temperatura TSD (u16)	reservado para F3
13–14	CRC16-Modbus (LE)	mesmo CRC já usado no app SBCCI

Usar o `alarm_sync` (já existente no controlador) em vez do `alarm` bruto, e enviar um pacote por medida concluída (não por temporizador), o que amarra cada pacote a uma varredura.

Como verificar

Teste da UART com decodificador em simulação; em placa, 24 h de captura sem lacunas de sequência e sem falhas de CRC.

C8. Posicionamento do sensor não reprodutível — Médio

Mudar qualquer outra parte do RTL desloca o atraso do caminho em nanossegundos.

Evidência

T5: sem trava, uma mudança no controlador e na UART alterou o atraso do caminho em 6,8 ns. Hoje só 3 FFs estão fixados, em nível de LAB, em três LABs diferentes; as 50 LUTs ficam livres.

Como corrigir

1. Compilar uma vez a versão de referência e extrair o posicionamento:

```
quartus_cdb max10_aging_port -c max10_aging_top --back_annotate=lc
```

2. **Cuidado**: esse comando grava *todas* as atribuições do projeto no QSF (570 no build atual). Filtrar apenas a hierarquia do sensor (cadeia, XOR, FF_sys, FF_ps, FF_alarm, `launch`) e salvar em um arquivo próprio, por exemplo `src/constraints/sensor_placement.qsf`, aplicado pelo `create_project.tcl`. Travar o projeto inteiro faz o fitter falhar na primeira mudança de RTL, pois os nomes dos nós mudam (isso aconteceu nesta revisão).
3. Colocar FF_sys e FF_ps no mesmo LAB, para que o *skew* entre eles seja mínimo e constante.
4. Comentar cada atribuição no QSF com sua justificativa (exigência do plano, F2).
5. Atenção a um detalhe prático: o QSF gerado pela GUI não termina com quebra de linha; scripts que acrescentam linhas devem inserir uma antes.

Limitação que não tem correção na edição Lite

O roteamento não pode ser fixado (T6). Com posicionamento travado, o atraso de setup variou 21 ps e o de hold 208 ps entre builds. Duas consequências práticas: (a) uma campanha de envelhecimento deve usar *um único* `.sof` do início ao fim — a reprodutibilidade entre builds só importa para comparar placas ou versões; (b) a migração do caminho para a cadeia de carry (Seção 5) reduz essa variação, porque a ligação entre estágios é fixa em silício.

Como verificar

Script de comparação: dez compilações com alterações fora do sensor produzem posições idênticas das células do sensor e dispersão do atraso do caminho (STA) menor que $\Delta/4$.

C9. Comentários e documentação incorretos — **Baixo**

Comentários errados induzem quem chega a decisões erradas.

- Topo: atribuição dos 96 ps ao Xilinx (C4); 179 ps em vez de 178,57 ps.
- SDC: margem do *catcher* "∼192 ps" (são 10 ns); trecho longo de narrativa de depuração no lugar de justificativas.
- Controlador: cabeçalho diz "avança" sem dizer que avançar significa `phaseupdown=0`; QSF comenta "`slack_count[7:0]`" com 4 displays.
- `hdl/README.md` e `hdl/src/README.md` ainda dizem "nada aqui é um design funcional".

5 O que pode ser melhorado

Melhorias, em ordem de relação valor/risco. As três primeiras são recomendadas para antes das campanhas.

5.1 M1. Voltar ao *dual-adder* da referência, na cadeia de carry

O port trocou o somador *ripple-carry* de referência (`u_sensor` dirigido por *toggle* + `u_canary` para erro funcional) por uma cadeia de inversores de outro artigo. Recomenda-se voltar ao *dual-adder*, implementado na cadeia de carry dedicada do LE (modo aritmético, `cin`/`cout`). Ganhos: (i) comparabilidade científica com o Artix-7, exigida pelo requisito 4; (ii) fiação entre estágios fixa em silício — T7 mediu dispersão de 68 ps entre builds *sem nenhuma trava*, contra 6,8 ns da cadeia de inversores livre; (iii) recupera o canário de erro funcional. Os `lut_mask` do somador da referência mapeiam diretamente: soma `16'h9696`, carry `16'hE8E8`.

5.2 M2. Estimar a borda por curva de probabilidade

Hoje (e também na referência) a FSM para no *primeiro* passo com alarme. Perto da borda o alarme é probabilístico (ruído, *jitter*, metaestabilidade). Em vez de parar, pode-se contar a fração de ciclos com alarme em cada fase e ajustar uma curva sigmoide (função erro); o centro da curva tem resolução *abaixo*

de um passo, mantendo a régua no cristal. É a melhoria que mais compensa os 96 ps, e custa apenas RTL e análise. O ganho real depende do ruído medido em placa.

5.3 M3. Dimensionar período e caminho juntos

O sinal de envelhecimento é proporcional ao atraso d do caminho. Com $T = 200\,\text{ns}$ e $d \approx 29\,\text{ns}$, desperdiça-se sinal e aproxima-se $d_{\min}$ do *catcher* (C3). A escolha deve seguir a regra de C3. Como referência, o somador de 16 bits em carry mediu 6,4 ns no corner lento; um clock de 100 MHz com *catcher* em ≈2,5 ns deixa folga nos dois lados e mantém a frequência de *toggle* da referência (o que também importa para o estresse AC: 5 MHz estressa 20× menos que 100 MHz).

5.4 M4. Confirmar cada passo de fase

O *handshake* temporizado está correto, mas não detecta um passo perdido. Como o pulso de `phasedone` cai e sobe dentro de um ciclo, um FF sensível à borda de descida (já existe, `pd_low_neg`) pode ser zerado a cada passo e verificado ao fim de `WAIT`; a falta de confirmação incrementa um contador de erro enviado na telemetria. Em simulação o *flag* global já acende (T9), o que indica que a abordagem funciona com o modelo da Intel.

5.5 M5. Mais sensores e mais régua

- Os contadores C3 e C4 do PLL estão livres: dois sensores adicionais em paralelo dão medidas independentes por varredura (reduzem σ por média) ou cobrem regiões diferentes do *die*.
- *Vernier* com dois PLLs (tarefa F1.4 do plano): resolução efetiva $|T_1 - T_2|$ mantendo a régua no cristal. Alto valor e alto risco; depende do *jitter* relativo medido em placa. O modelo oficial do PLL permite prototipar em simulação primeiro.

5.6 M6. Operação e instrumentação

- A lógica do MAX 10 roda em CRAM (SRAM), carregada da *flash* interna só ao ligar. Gravando o `.sof` por JTAG a cada sessão (como o App_2Nexys já faz), a retenção da *flash* sob calor deixa de ser crítica; o CRC de detecção de erro da CRAM (`ERROR_CHECK_FREQUENCY_DIVISOR`, já no QSF) monitora corrupção.
- Integrar o ADC em modo diodo térmico (TSD) cedo, mesmo sem calibração, para que todo dado de placa já venha com temperatura.
- Expor no pacote os parâmetros de build (passo, faixa, hash do commit), tornando cada CSV autodescritivo.

6 Próximas tarefas e distribuição na equipe

6.1 Princípio da divisão

Para trabalhar em paralelo sem colisão, a equipe precisa primeiro congelar três *contratos* (semana 1), e depois cada pessoa trabalha contra o contrato, não contra o código das outras:

1. **Interface do núcleo do sensor**: portas e parâmetros de topo (`STEP_PS`, `MAX_STEPS`, `ALARM_FILTER_SAMPLES`, sinais de resultado, validade e sequência).
2. **Pacote de telemetria v2** (C7): documento em `max10_port/docs/` — permite que o *host* seja escrito antes do RTL estar pronto.
3. **Fluxo de build e teste**: `create_project.sh`, `build_bitstream.sh`, `run_sim.sh` — todo commit precisa passar neles.

6.2 Papéis

Tabela 4: Distribuição sugerida (4 a 6 pessoas; papéis podem ser acumulados).

Papel	Frente	Tarefas	Depende de
P1 Infraestrutura	F2	C1 (higiene, scripts, versão fixada); C8 (script de extração e comparação de posicionamento); script de regressão.	— (começa já)
P2 RTL do sensor	F1	C2 (FSM contínua); C3 (*catcher*, detector de saturação, `set_max_delay`); C6 (filtro); M4 (confirmação de passo); parametrização.	contrato 1
P3 Verificação	F1	C5 (suíte em três níveis); testes de cada correção de P2; extração automática do atraso do STA para a simulação.	contrato 1; P1
P4 Régua e bancada	F1/F5	C4 (IP a 96 ps); ensaios H1–H5 em placa; varredura do *catcher* em placa; M3 (escolha de T e caminho).	P1; P2 para H3+
P5 Host e dados	F4/F6	Receptor Python do pacote v2, CSV com sequência e metadados, gráficos; começa com um gerador de pacotes simulado.	contrato 2
P6 PVT e viabilidade	F3/E1	ADC TSD no FPGA (campo de temperatura do pacote); cálculo E1 de horas de estresse para um degrau acima de 3σ; M1 (*dual-adder* em carry).	contrato 1; dado σ de P4

6.3 Sequência

Tabela 5: Sequência de referência em semanas (ajustar ao calendário real).

Semana	Entregas	Marco
1	Contratos 1–3 escritos e revisados; C1 concluído (clone limpo compila); versão do Quartus fixada.	Base reproduzível
2	C4 (96 ps no fit); suíte de testes nível 1–2 rodando no estado atual (testes de C2/C3 *falhando*, como esperado); *host* lendo pacotes simulados.	Rede de testes pronta
3	C2, C3, C6, C7 implementados, cada um com seu teste passando; C8 (posicionamento travado e verificado).	RTL candidato
4	H1–H2 em placa (*bring-up* e medição do passo); *host* gravando CSV real.	Régua medida
5	H3 (repetibilidade, σ em contagens); E1 com o σ medido.	**Decisão go/no-go científica**
6	H4 (rastreamento sob variação de temperatura)	**Gate A**

Regras de convivência no repositório

Trabalhar sempre no clone `git` (nunca em cópias `.zip` — o incidente registrado no cabeçalho do topo nasceu disso); commits pequenos, com mensagem que diga o *porquê*; cada mudança de RTL acompanhada de teste; revisão por outra pessoa antes de mesclar; atas semanais em `max10_port/minutes/`, como o plano exige.

7 Como aferir que as implementações estão corretas

A aferição é organizada em níveis. Cada nível só tem valor se o anterior estiver verde, e cada critério é numérico e automatizável sempre que possível.

Tabela 6: Níveis de aferição.

Nível	Ferramenta	Critério de aceite
L0 Estático	`check_layout.sh`, Verilator `-lint-only`	Sem erros; sem portas desconectadas no relatório de conectividade do Quartus.
L1 Unitário	Icarus / Questa	Controlador, filtro, UART: todos PASS, incluindo casos de borda probabilística, fora de faixa e perda de *lock*.
L2 Integração	Questa + ALTPLL oficial	Contagem $= \lfloor (T-d)/\Delta \rfloor + 1$ para d em todos os corners (do STA) e com envelhecimento simulado; retorno de fase a 0; detector de saturação correto.
L3 Implementação	Quartus (fit, STA, script de comparação)	PLL com o passo pedido; restrição do *catcher* fechando; dez builds com posicionamento idêntico do sensor e dispersão de atraso $< \Delta/4$.
L4 Bancada	DE10-Lite, osciloscópio, forno	Ensaios H1–H6 abaixo.

7.1 Referência já obtida para o nível L2

A Tabela 7 é o resultado de T9/T10 no RTL atual e serve de linha de base: depois das correções, as linhas de 9 e 6 ns devem passar a seguir a fórmula (ou ser marcadas como saturadas), em vez de devolver 1065 silenciosamente.

Tabela 7: Simulação de laço fechado com o modelo oficial do PLL (RTL atual, $\Delta = 178{,}57$ ps).

Atraso d	Previsto $\lfloor (T-d)/\Delta \rfloor + 1$	Obtido	Situação
29,2 ns	957	957	correto (corner lento do STA)
20,0 ns	1009	1009	correto
12,0 ns	1053	1053	correto
10,5 ns	1062	1062	correto, margem de 0,5 ns (corner rápido)
9,0 ns	1070	**1065**	**saturado** — defeito C3
6,0 ns	1087	**1065**	**saturado** — defeito C3

7.2 Ensaios em bancada (L4)

H1 — *Bring-up*
LEDs: *lock* aceso, calibração concluindo, contagem nos displays. *Host* recebe pacotes v2 sem erro de CRC. **Critério:** 10 minutos sem falha.

H2 — Medição do passo
Rotear c0 e c1 para pinos GPIO; com osciloscópio, medir a defasagem após K passos ($K = 100 \Rightarrow \approx$ 9,6 ns, fácil de medir). $\Delta =$ defasagem$/K$; repetir para vários K para obter a não linearidade. **Critério:** dentro de $\pm 2\%$ do nominal; a distribuição vai para o relatório de caracterização da régua (entregável F1).

H3 — Repetibilidade
$\geq$1000 varreduras com temperatura estável. **Critério:** histograma unimodal; σ em contagens registrado — este é o número que alimenta a decisão go/no-go.

H4 — Rastreamento
Variar a temperatura do forno em degraus e manter cada patamar. **Critério (Gate A):** a contagem acompanha a temperatura de forma monotônica e reprodutível na subida e na descida, sem perda de rastreamento.

H5 — Fronteira do *catcher*
Varrer C2 dinamicamente até a saturação aparecer. **Critério:** a fronteira fica a uma distância maior que a margem de projeto em todos os corners térmicos ensaiados.

H6 — Estabilidade de longa duração
24–72 h contínuas. **Critério:** sem lacunas de sequência, sem perda de *lock*, contador de erros de passo em zero.

O critério de go/no-go científico

Com σ (H3) e Δ (H2) medidos, o menor degrau detectável é $3\sigma\Delta$. A trilha E1 estima quantas horas de estresse, no envelope térmico admissível, produzem esse degrau em 55 nm. Se o tempo for viável, o projeto segue para campanhas; se não, M2 e o *Vernier* passam a ser o caminho principal antes de qualquer campanha longa — e, no limite, o resultado é o estudo de limite de detectabilidade proposto em `nota-limite-deteccao.pdf`, que o plano já aceita como desfecho válido.

8 Limitações desta revisão

- Nenhum ensaio em placa foi feito: passo real, *jitter*, metaestabilidade e comportamento real do `phasedone` no silício só são conhecidos pelos ensaios H1–H5.
- A simulação de integração é determinística: usa o atraso do STA e não modela ruído. Ela valida a lógica do laço, não a estatística do sensor.
- As medições de reprodutibilidade (T4, T5, T7) foram feitas com o projeto atual e com um somador de teste; os números da versão final precisam ser refeitos no nível L3.

A Glossário Vivado ↔ Quartus

Vivado / Artix-7	Quartus / MAX 10
`LUT3_L #(.INIT(...))`	`fiftyfivenm_lcell_comb #(.lut_mask(16'h...))` (LUT de 4 entradas)
`DONT_TOUCH`, `KEEP`	parâmetro `dont_touch` do átomo; atributos `keep`, `preserve`, `noprune`
`LOC`/`BEL`	`set_location_assignment LCCOMB_X.._Y.._N..` ou `FF_X.._Y.._N..`
Pblock / `AREA_GROUP`	região LogicLock
`FIXED_ROUTE`	sem equivalente na edição Lite
`write_checkpoint` + extração de LOC	`quartus_cdb -back_annotate=lc`
MMCM, `PSEN`/`PSINCDEC`/`PSDONE`	ALTPLL, `phasestep`/`phaseupdown`/`phasedone`, `phasecounterselect`
`psincdec=0` (adianta)	`phaseupdown=0`
XADC (temperatura e VCCINT)	ADC em modo TSD (só temperatura; VCCINT exige modificar a placa)
Device View	Chip Planner
`report_timing`	`quartus_sta`, `get_timing_paths`
Simulação pós-implementação com SDF	não disponível para MAX 10 (só netlist funcional)

B Ambiente e comandos

Na máquina do laboratório, o script `~/.bashrc.d/intel-fpga.sh` coloca o Quartus 25.1std e o Questa 2025.2 no `PATH` e configura a licença do Questa (variável `SALT_LICENSE_SERVER`; o Questa 2025 não aceita a licença só pela `LM_LICENSE_FILE`). `intel-fpga status` mostra a versão ativa; `intel-fpga 21.1` volta à instalação do laboratório. Comandos usados nesta revisão:

```
# compilacao completa e relatorios
quartus_sh --flow compile max10_aging_port -c max10_aging_top
grep -a -E 'Nominal VCO|VCO phase shift step' output_files/max10_aging_top.fit.
    rpt

# extracao de posicionamento (filtrar depois so o sensor!)
quartus_cdb max10_aging_port -c max10_aging_top --back_annotate=lc

# atraso do caminho do sensor via Tcl do timing analyzer
quartus_sta -t path.tcl # usa get_timing_paths -from launch -to *ff_sys_clk_q

# simulacao com o modelo oficial do PLL e com os atomos MAX 10
vsim -c -L altera_mf_ver -L altera_ver -L altera_lnsim_ver top_tb +DELAY_PS=29246
vsim -c -L fiftyfivenm_ver chain_tb
```

C Arquivos dos experimentos

Os experimentos foram executados fora do repositório, em uma área temporária da sessão de revisão. Os arquivos relevantes devem ser incorporados à suíte de testes (tarefa de P3):

- `sim/edge_tb.sv` — controlador com borda probabilística (T11 e base para C6).
- `sim/pll/pll_tb.sv` — caracterização do ALTPLL em simulação (T2).
- `sim/top/top_tb.sv`, `sim/top/chain_beh.sv` — laço fechado com PLL oficial (T9, T10).

- `carry/carry_top.sv` — somador em cadeia de carry (T7).
- `path.tcl` — extração do atraso do caminho pelo STA (T5, T7).